Deep Dive into Deep Learning for Time Series Imputation

How AI reconstructs missing data, and why getting those estimates right matters

Imagine monitoring a patient's heart rate, tracking energy consumption across a city, or measuring temperature in a factory. The sensors collect readings every few seconds, but some values inevitably go missing. A device may fail, a network connection may drop, or measurements may arrive irregularly. Those gaps can distort analysis and weaken the predictions built on top of the data.

This is the problem addressed by multivariate time-series imputation (MTSI): estimating missing values in time-dependent data that contains multiple related variables. The survey *Deep Learning for Multivariate Time Series Imputation: A Survey* maps the deep-learning methods researchers use for this task. It reviews the field rather than proposing one new algorithm that solves every case.

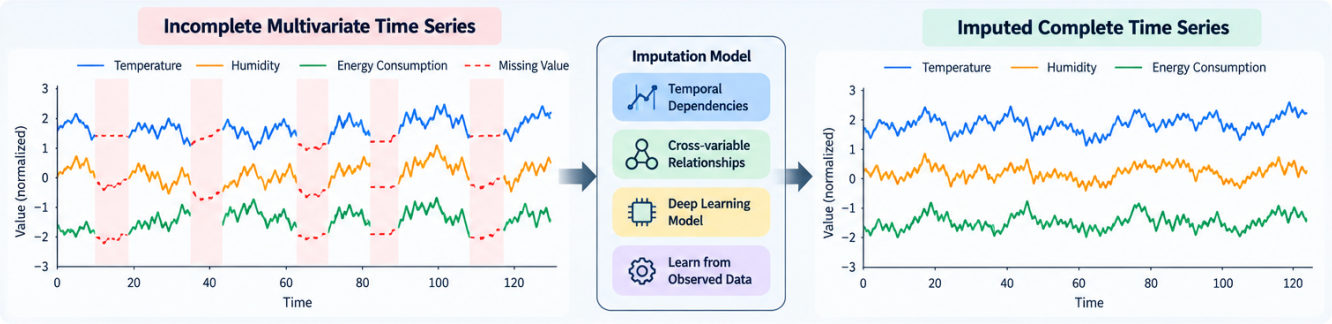


Figure 1: Illustrative example of multivariate time series imputation. The left plot shows an incomplete multivariate time series with missing values (dashed segments). An imputation model leverages temporal dependencies and cross-variable relationships to reconstruct the missing values, producing a complete and consistent time series on the right.

Figure 1. Multivariate time-series imputation in practice. When observations are missing, an imputation model uses patterns over time and relationships between variables to estimate the gaps. The completed series is more useful for analysis, but estimated values should not be mistaken for observed measurements.

Why missingness is more complicated than it looks

A missing value is not always just an empty cell. The reason it is missing can affect which method is appropriate. The survey discusses three common mechanisms. **Missing completely at random (MCAR)** means missingness is independent of the data. **Missing at random (MAR)** means the chance of a value being missing depends on observed information. **Missing not at random (MNAR)** means missingness can depend on the unobserved value itself. MNAR is especially difficult: a sensor may fail more often under extreme conditions, precisely when its measurements matter most.

A model can produce a smooth-looking series and still be wrong. The important question is not only whether the gaps are filled, but whether the estimates preserve the patterns that matter for the analysis or prediction that follows.

Three broad families of approaches

The survey groups methods by their modeling approach and the kind of output they produce. Predictive methods estimate missing values directly; generative methods model plausible completions and their uncertainty; large-model approaches adapt pretrained time-series or language models.

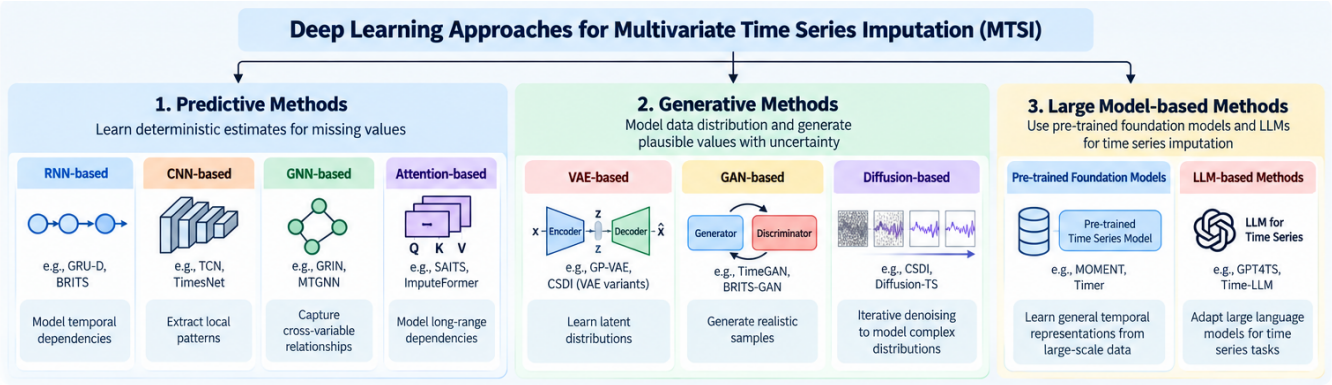


Figure 2: Taxonomy of deep learning methods for multivariate time series imputation. The survey categorizes existing approaches into predictive methods, generative methods, and large model-based methods. Each category includes representative architectures and their core ideas for handling missing values in multivariate time series data.

Figure 2. Main families of deep-learning methods for time-series imputation. Predictive models estimate missing values directly. Generative models learn distributions that can represent more than one plausible completion. Large-model approaches reuse pretrained representations, although adapting them to numerical time series remains an active research problem.

1. Predictive methods: estimate the missing value

Predictive methods produce a fixed estimate for each missing entry. Recurrent neural networks (RNNs), including GRU-based approaches and BRITS, process sequences and learn how values evolve over time. Some methods account explicitly for missingness patterns or use information from both past and future observations. Their strength is temporal modeling, although sequential processing can limit efficiency on long sequences.

Convolutional neural networks (CNNs) capture local patterns in windows of observations. Graph neural networks (GNNs) represent relationships among variables or sensors as a graph, which can help when related measurements inform one another. Attention-based methods, such as SAITS and ImputeFormer, can connect distant time steps and variables. That flexibility can come at the cost of memory and computation as sequences grow.

2. Generative methods: model plausible alternatives

Generative methods approach imputation as a distribution-learning problem. Variational autoencoders (VAEs), including GP-VAE, learn latent representations from which plausible data can be reconstructed. Generative adversarial networks (GANs) train a generator against a discriminator, but their training can be unstable. Diffusion models, including CSDI and related methods, gradually denoise samples to generate plausible completions. They can represent uncertainty, but repeated denoising steps can be computationally expensive.

This is useful when a single point estimate is not enough. Several values may reasonably fill the same gap, and a probabilistic model can help express that ambiguity. Its uncertainty estimates still need careful evaluation.

3. Large-model approaches: reuse learned representations

A newer direction adapts pretrained time-series foundation models, such as MOMENT and Timer, or experiments with large language models for time-series tasks. Pretraining may provide reusable temporal representations and reduce the need to learn every pattern from scratch. But time series are not ordinary text: scale, sampling frequency, variable relationships, and domain-specific behavior all matter. A large model does not automatically produce better imputations.

A quick comparison

Family	Examples	Strength	Limitation
RNN / recurrent	GRU-D, BRITS	Temporal patterns	Sequential processing can be slow on long sequences
CNN	Temporal convolutions	Local patterns and efficient windows	May miss long-range relationships
GNN	GRIN	Relationships among variables or sensors	Graph design and computation can be complex
Attention	SAITS, ImputeFormer	Long-range dependencies	Memory and compute cost
VAE / GAN / diffusion	GP-VAE, GAN variants, CSDI	Plausible samples and uncertainty	Training complexity; diffusion can be expensive
Foundation / LLM	MOMENT, Timer, GPT4TS	Reuse of pretrained representations	Adaptation and evaluation remain challenging

Table 1. A comparison of representative model families. Their performance depends on the dataset, the missingness mechanism, and how the task is evaluated.

From papers to practice: benchmarks and toolkits

The survey highlights resources intended to make experimentation more reproducible. PyPOTS provides tools for partially observed time series, including implementations of imputation methods. TSI-Bench provides benchmark pipelines and datasets for comparing approaches. These resources help researchers compare methods under more consistent data splits, masking strategies, preprocessing, and metrics.

What remains difficult?

Realistic missingness. Many evaluations create artificial gaps, but real-world missingness may be structured or MNAR. Methods need testing under conditions that resemble actual sensor and operational failures.

Downstream usefulness. Low reconstruction error does not always mean better forecasting, classification, or anomaly detection. Imputation should be assessed both on the recovered values and on the tasks that use them.

Scale and uncertainty. Long sequences, many variables, and large models can make computation costly. Systems should also communicate when estimates are uncertain instead of presenting every reconstructed value as equally trustworthy.

Generalization. Models trained on one domain may not transfer to another with different sampling rates, variable relationships, or operating conditions. Multimodal context and domain-aware adaptation are promising directions, but they need rigorous validation.

The key takeaway

The survey points to three useful directions rather than one universal winner. Predictive models provide direct estimates; generative models can represent multiple plausible completions; and pretrained large models offer a way to reuse learned representations. The right choice depends on the data, why values are missing, the cost of errors, and the task that follows.

For me, the most important lesson is that filling a gap is not the same as recovering the truth. A model's output should be judged by whether it preserves useful temporal and cross-variable patterns, holds up under realistic missingness, and helps the downstream task. A complete-looking dataset is only a starting point.